Phase 5: HFT Order book

100 Points Possible

5/4/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

Details

Phase 5: High-Frequency Trading (HFT) Order Book Implementation & Performance Optimization

Overview

Welcome to Phase 5 of the class project! In this phase, **you will build a high-performance order book**, simulating real-world high-frequency trading (HFT) conditions. Your goal is to create an efficient, scalable, and optimized order book that can process **millions of orders per second** while maintaining **low latency**.

You will:

- Implement an order book with **string-based order IDs**.
- Perform **rigorous performance analysis** and measure **execution time**.
- Optimize the order book to achieve **maximum speed and efficiency**.
- Conduct **stress tests using generated orders** to evaluate scalability.
- Submit a **report with execution time results and a performance chart**.

Step 1: Implementing the Order Book

1.1 Core Data Structures

Your order book must efficiently store and manage orders using:

- `std::map<double, std::unordered_map<std::string, Order>>` → Maintains **sorted price levels** for bids/asks.
- `std::unordered_map<std::string, Order>` → Provides **fast lookup** using string-based order IDs.

Example Implementation

```
#include <iostream>
#include <map>
#include <unordered_map>
#include <string>

struct Order {
    std::string id;  // String-based order ID
    double price;
    int quantity;
    bool isBuy;
};

class OrderBook {
private:
    std::map<double, std::unordered_map<std::string, Order>> orderLevels;
    std::unordered_map<std::string, Order> orderLookup;

public:
    void addOrder(const std::string& id, double price, int quantity, bool isBuy) {
        Order order = {id, price, quantity, isBuy};
        orderLevels[price][id] = order;
        orderLookup[id] = order;
    }
};
```

✅ **Orders are efficiently stored and quickly accessible.**

1.2 Adding, Modifying, and Deleting Orders

Your order book must support:

- **Adding orders**
- **Modifying orders efficiently**
- **Deleting orders without unnecessary overhead**

Example Code

```
void modifyOrder(const std::string& id, double newPrice, int newQuantity) {
    if (orderLookup.find(id) != orderLookup.end()) {
        Order oldOrder = orderLookup[id];
        orderLevels[oldOrder.price].erase(id);
        addOrder(id, newPrice, newQuantity, oldOrder.isBuy);
    }
}

void deleteOrder(const std::string& id) {
    if (orderLookup.find(id) != orderLookup.end()) {
        Order order = orderLookup[id];
        orderLevels[order.price].erase(id);
        orderLookup.erase(id);
    }
}
```

✅ **Ensures efficient order modification without unnecessary overhead.**

Step 2: Performance Analysis & Bottleneck Identification

2.1 Profiling Execution Time

Use **chrono** to measure execution speed:

```
#include <chrono>

void benchmark() {
    auto start = std::chrono::high_resolution_clock::now();

    addOrder("ORD001", 50.10, 100, true);

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end - start;

    std::cout << "Execution time: " << elapsed.count() << " seconds" << std::endl;
}
```

✅ **Identifies slow operations for optimization.**

2.2 Detecting Bottlenecks

- **Are lookups slow?** Optimize **hash functions** to reduce collisions.
- **Is memory usage high?** Use **memory pools** instead of dynamic allocation.
- **Are branching delays slowing execution?** Apply **branch prediction techniques**.

Step 3: Advanced Optimization Techniques

3.1 Memory Management

Preallocate memory to avoid frequent allocations:

```
class OptimizedOrderBook {
    std::vector<Order> orderPool;

public:
    OptimizedOrderBook(size_t size) { orderPool.reserve(size); }
};
```

✅ **Prevents slow dynamic allocations** during runtime.

3.2 Speed Enhancements using Loop Unrolling

```
void processOrders(std::vector<Order>& orders) {
    for (size_t i = 0; i < orders.size(); i += 2) {  // Unrolling
        handleOrder(orders[i]);
        if (i + 1 < orders.size()) handleOrder(orders[i + 1]);
```

```
    }
}
```

✅ **Processes multiple orders per iteration for efficiency.**

3.3 Lock-Free Data Structures

Instead of using **mutexes**, use **atomic operations** for high-speed execution:

```
#include <atomic>

std::atomic<int> orderCount(0);

void addAtomicOrder() {
    orderCount.fetch_add(1, std::memory_order_relaxed);
}
```

✅ **Eliminates synchronization delays.**

Step 4: Unit Testing & Stress Testing

4.1 Unit Testing for Accuracy

Use **assertions** to verify correct behavior:

```
#include <cassert>

void testAddOrder() {
    OrderBook book;
    book.addOrder("ORD001", 50.10, 100, true);

    assert(book.orderLookup.count("ORD001") == 1);  // Order should exist
}
```

✅ **Prevents errors from going unnoticed during development.**

4.2 Stress Testing with Generated Orders

Code to generate orders for stress testing

```
#include <random>

void stressTest(OrderBook& book, int numOrders) {
    std::mt19937 rng(std::random_device{}());
    std::uniform_real_distribution<double> priceDist(50.0, 100.0);
    std::uniform_int_distribution<int> quantityDist(1, 500);

    for (int i = 0; i < numOrders; ++i) {
        std::string id = "ORD" + std::to_string(i);
        double price = priceDist(rng);
        int quantity = quantityDist(rng);

        book.addOrder(id, price, quantity, true);
    }
}
```

✅ **Tests scalability with randomized order flow.**

Step 5: Execution Time Chart

Measure execution time over different order volumes

Once stress testing is complete, gather execution time data at different scales. Use Python's **matplotlib** to generate a chart.

```
import matplotlib.pyplot as plt

order_sizes = [1000, 5000, 10000, 50000, 100000]
execution_times = [0.002, 0.015, 0.030, 0.120, 0.250]  # Replace with actual benchmarks

plt.plot(order_sizes, execution_times, marker='o', linestyle='-', color='b')
plt.xlabel("Number of Orders")
plt.ylabel("Execution Time (seconds)")
plt.title("HFT Order Book Performance")
plt.grid()
plt.show()
```

✅ **Visualizes performance scalability.**

Step 6: Deliverables

1️⃣ Source Code

Submit your **full order book implementation**, including optimizations.

2️⃣ Performance Analysis Report

Benchmark results detailing:

- **Execution time comparisons**
- **Optimization effectiveness**
- **Latency breakdowns**

3️⃣ Unit Test & Stress Test Results

Provide test case outputs proving **accuracy and stability** under load.

4️⃣ Video Demonstration

Create a video showcasing:

- **Real-time order processing**
- **Performance visualizations**

- **Scalability results under extreme conditions**

Conclusion

This phase mirrors the challenges faced by **real-world HFT firms**, focusing on **efficient order processing, extreme speed, and rigorous testing**. By completing this project, **you will gain practical experience** in high-performance trading systems.

Keep in mind, this submission will count for everyone in your Q22026 Group group.

Choose a submission type

Text

Upload

More

Submit Assignment